

Decomposing Weights into Human Concepts

Dictionary learning on bilinear MLP weights yields a human-programmable visual vocabulary

Pranav Viswanath | 2026

Abstract

MNIST is one of the most classic benchmarks in machine learning (LeCun et al., 1998), and despite how well-studied it is, we still don't fully understand how a trained model computes its output: the nonlinearity in standard MLPs makes the weights uninterpretable in isolation. The exciting approach pioneered by Doods and Gauderis (2024) is to replace the activation function with a bilinear layer, which makes each output logit an exact quadratic form in the input, fully determined by the weights. That means we're not looking at millions of activations post-hoc; we're looking at the weights directly, because the weights are all a model actually is. This project applied several tensor decomposition methods to a bilinear MLP trained on MNIST and found that dictionary learning (Mairal et al., 2010; Olshausen and Field, 1997) recovers human-interpretable concepts like strokes, curves, and loops, so interpretable that I was able to reconstruct the full classifier by hand using a human programming language, composing these learned features, and recovered 90.1% test accuracy against the model's trained 96.8%.

Why bilinear, why weights

The standard MLP applies a pointwise nonlinearity (ReLU, GELU, etc.) after each linear layer. This is what makes the weights hard to analyze: you can't say anything about what a weight matrix does without knowing what activations look like for a given input, and that's distribution-dependent. The dominant response has been activation-based interpretability: train sparse autoencoders (SAEs) on internal representations (Elhage et al., 2022; Templeton et al., 2024). That's genuinely powerful, but it's fundamentally post-hoc and distribution-bound; the features you find change with the data you probe on (Friedman et al., 2025).

The whole point of the bilinear MLP (Doods and Gauderis, 2024) is to replace the activation function with element-wise multiplication, $\mathbf{h} = (W_L \mathbf{x}) \odot (W_R \mathbf{x})$, which makes the output logit for class c collapse into a symmetric quadratic form:

$$\text{logit}_c = \mathbf{x}^\top B_c \mathbf{x}, \quad B_c = \frac{1}{2} \sum_o u_{co} (\mathbf{w}_o^L \mathbf{w}_o^{R\top} + \mathbf{w}_o^R \mathbf{w}_o^{L\top})$$

The model's full computation for class c is captured by a single symmetric 784×784 matrix B_c : no activations, no approximation. All ten together form the weight tensor $\mathcal{B} \in \mathbb{R}^{10 \times 784 \times 784}$, the complete and exact description of the classifier. We train this to 96.8% validation accuracy using AdamW with Gaussian noise augmentation ($\sigma = 0.4$).

Decomposition methods: what worked and why

The question is how to factorize $\mathcal{B}$. CP decomposition (Harshman, 1970; Kolda and Bader, 2009) ($\mathcal{B}[c, i, j] \approx \sum_r L_{ir} R_{jr} D_{cr}$, rank 64) already recovers human-readable stroke detectors

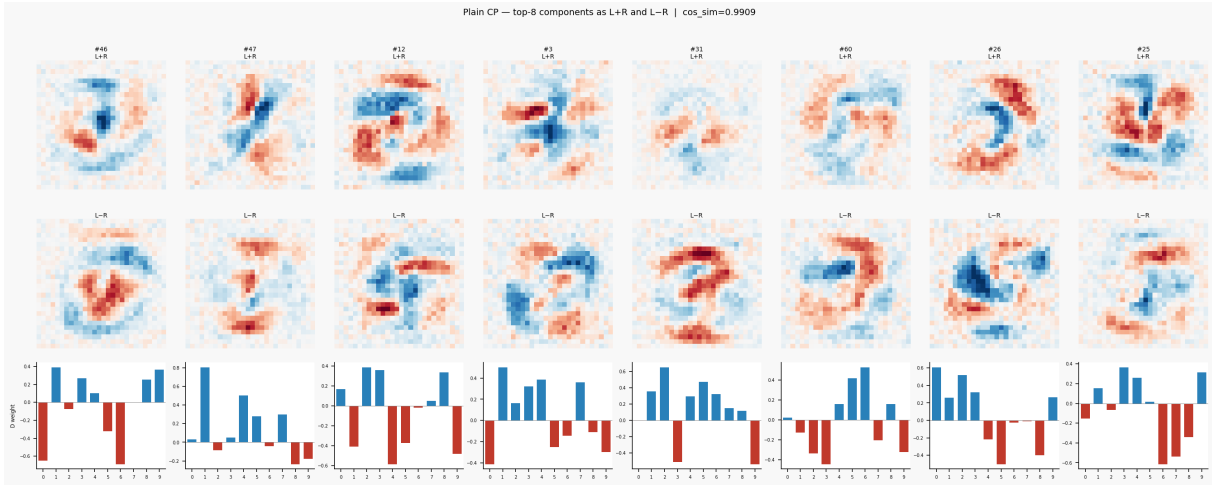


Figure 1: **Plain CP decomposition (rank 64) already finds stroke detectors in the weight tensor.** Top two rows: symmetric ($L + R$) and antisymmetric ($L - R$) pixel-space images for the top-8 components. Bottom: class-loading vector D_r per component; every component serves multiple digit classes simultaneously. Kruskal uniqueness is verified ($k_L + k_R + k_D = 138 \geq 130$): this is the *only* rank-64 factorization of $\mathcal{B}$ (Kruskal, 1977). Cosine similarity to original weight tensor: 0.9909.

with 99.1% reconstruction fidelity and 96.5% task accuracy (Figure 1). Crucially, Kruskal’s theorem (Kruskal, 1977) tells us this factorization is *algebraically unique*: with Kruskal ranks $k_L = k_R = 64$, $k_D = 10$, and $138 \geq 2(64) + 2 = 130$ (confirmed by full order of magnitude), no alternative factorization exists. The cross-digit sharing isn’t a coincidence of initialization; it’s geometrically forced by the tensor itself.

From there I tested a sweep of priors to see if anything could sharpen interpretability further:

- **L1 on D** : rescales weights uniformly; normalization during visualization undoes it completely. No visual change.
- **Non-negativity**: the model relies on signed contrastive codes, and forbidding negative values collapses accuracy to 47%.
- **Hoyer sparsity** (Hoyer, 2004): concentrates pixel magnitudes but destroys spatial coherence ($0.47 \rightarrow 0.05$). Turns out pixel-level sparsity and feature-level coherence are totally orthogonal objectives.
- **HOSVD**: perfect reconstruction but forces eigenvector orthogonality, producing full-image swirling patterns that don’t correspond to any visual primitive.

Spatial coherence saturated around 0.50 across every CP variant. The insight: the rank-1 outer product constraint is the ceiling. Loops and curves are full-rank features: you can’t represent a closed loop as a single outer product without decomposing it into sign-opposing components. That’s why we need full-rank atoms.

Dictionary learning: a vocabulary of six atoms

Dictionary learning (Mairal et al., 2010; Olshausen and Field, 1997) replaces rank-1 components with arbitrary-rank atoms: $B_c \approx \sum_k \alpha_{ck} A_k$, where each atom $A_k \in \mathbb{R}^{784 \times 784}$ has rank 24–32 and α_{ck} is a sparse code (L1 regularization $\alpha = 0.1$, optimized jointly).

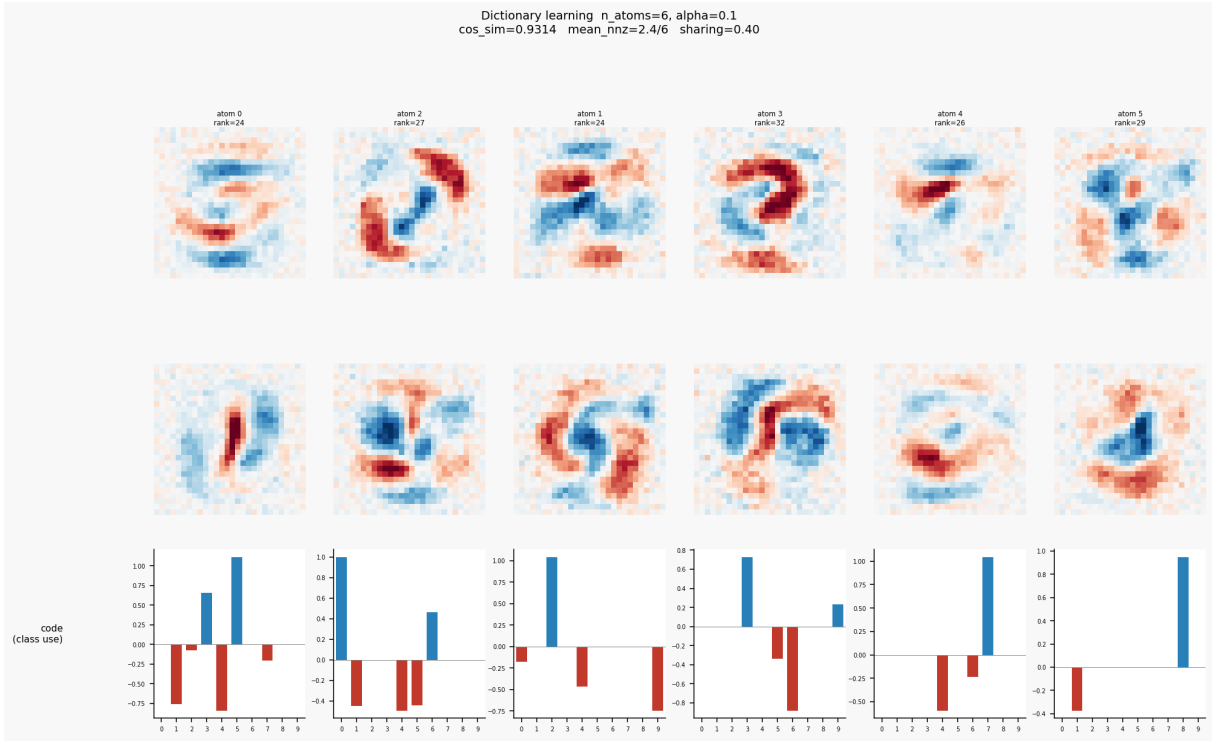


Figure 2: **The six atoms from dictionary learning ($K = 6$, $\alpha = 0.1$).** For each atom, the top two rows show the leading eigenvectors of the 784×784 atom matrix in pixel space (red = positive, blue = negative). The bottom row shows each atom’s code vector α_k across digit classes; each atom appears in multiple digits with both signs, forming a contrastive basis rather than class-specific detectors. Cosine similarity to weight tensor: 0.9314. Mean codes per class: 2.4 (genuine cross-class sharing). Cf. [Dooms and Gauderis \(2024\)](#), Appendix B.

The number of atoms reveals a sharp phase transition. At $K = 6$, each atom is used by 2.4 digit classes on average: the model shares a vocabulary across digits. At $K = 10$, the sharing ratio drops to 0.10 and each digit gets its own dedicated atom: memorization, not generalization. Six is the natural compression point.

The six atoms sort themselves into named visual primitives: *upper-curve*, *open-curve*, *closed-loop*, *hook-end*, *diagonal*, *double-loop* (Figure 2). These names aren’t imposed: they come from visual inspection of each atom’s leading eigenvector projected into pixel space. Every digit in MNIST decomposes into 2–3 of these primitives.

Composing the atoms: a programming language for digits

Here’s the experiment. Given only the six atom images (without looking at a single digit image), specify the code vector $\alpha_c \in \mathbb{R}^6$ for each digit class using visual reasoning alone. Then rebuild all ten B_c matrices as $\sum_k \alpha_{ck} A_k$ and run the classifier on the 10,000-image test set.

The reasoning reads directly from the atoms. **Digit 0**: pure closed-loop (atom 2, +1.0; nothing else). **Digit 1**: suppress everything: no loops, arcs, or hooks. **Digit 8**: double-loop atom alone. **Digit 6** is the most instructive: it has a closed loop (atom 2, +0.6), an upper arc (atom 0, +0.2), and *explicitly no hook*, so atom 3 fires negatively (−0.8). That suppressed hook is exactly what distinguishes a 6 from a 9. Figure 3 shows this decomposition laid out; the story reads directly from the weights.

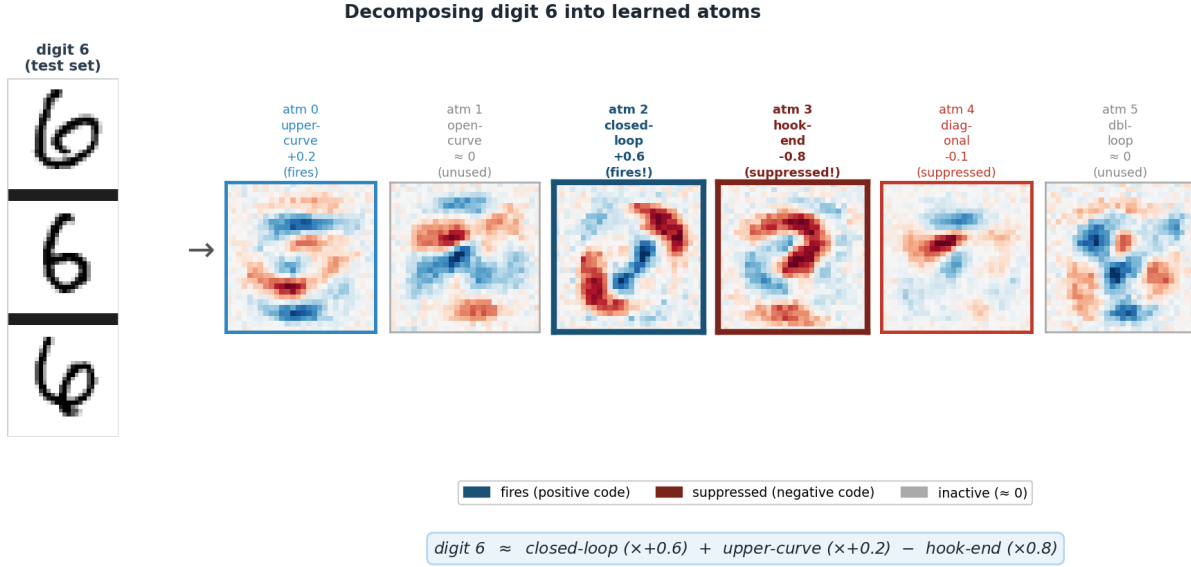


Figure 3: **Decomposing digit 6 into its learned atoms.** Left: three test-set examples of digit 6. Right: the six atoms annotated with their hand-specified code weights. **Blue** borders = fires (positive); **dark red** = suppressed (negative); gray = inactive. The recipe: closed-loop (+0.6) + upper-curve (+0.2) – hook-end ($\times 0.8$).

I specified all ten code vectors this way and ran the classifier. The result: **90.1% test accuracy** vs. the model’s trained **96.8%** (Figure 4). Nine of ten digits are recovered near-perfectly. The only failure is digit 9 ($\approx 50\%$ hand-written accuracy): its correct code requires suppressing the open-curve while activating a subtle hook, a nuance the learned weights encode but that pure visual inspection misses. Everything else is readable by eye.

This isn’t model compression or a post-hoc explanation. The code vectors were derived from the weight atoms alone, without access to any input image. The result is a set of human-readable programs: “if it has a loop and no hook, it’s a 6,” written directly from the weights.

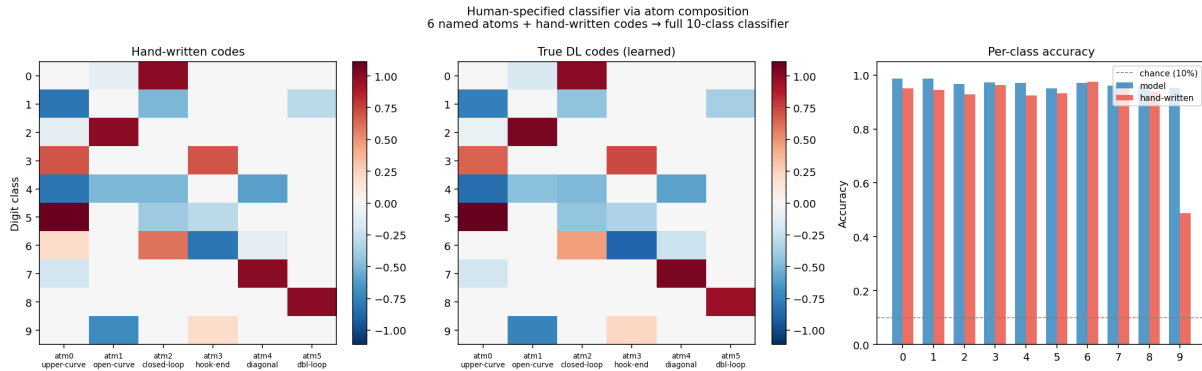


Figure 4: **Human-specified classifier via atom composition.** Left: hand-written code vectors (visual inspection only, no data). Center: the model’s learned codes (note structural agreement for 9/10 digits). Right: per-class test accuracy, trained model (blue) vs. hand-specified (red); dashed = 10% chance. Overall: **90.1%** hand-written vs. **96.8%** model. Digit 9 is the sole failure.

What this means

A bilinear MLP trained on MNIST stores a visual programming language in its weights: six atoms, ten code vectors, 90% accuracy from visual inspection alone. The decomposition is the unique algebraic factorization of the exact weight tensor (Kruskal, 1977): not approximate, not post-hoc. The same architecture generalizes beyond MNIST: Doods et al. (2024) apply it to Qwen3-0.6B and find circular manifolds for temporal tokens in the weight tensor.

This is the same pattern GoodFire demonstrated at biological scale: decomposing a model trained to predict DNA fragment length revealed Alzheimer’s biomarker structure the model had learned without supervision. Distilling that into a human classifier outperformed existing predictors. The lesson in both cases: models learn vocabularies. When the architecture lets you read the weights, you can inherit them. We don’t need to know what a model is doing. We need to make it the kind of model whose weights can tell us.

References

- T. Doods and W. Gauderis. Bilinear MLPs enable weight-based mechanistic interpretability. *arXiv preprint arXiv:2410.08417*, 2024. Accepted at ICLR 2025.
- T. Doods, W. Gauderis, et al. Finding manifolds with bilinear autoencoders. *arXiv preprint arXiv:2510.16820*, 2024.
- N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, D. Hernandez, D. Drain, E. Hubinger, J. Kernion, B. Lasenby, et al. Toy models of superposition. https://transformer-circuits.pub/2022/toy_model/, 2022.
- J. Friedman, A. Variengien, T. Conerly, T. Bricken, J. Lindsey, and A. Templeton. Are sparse autoencoders useful? A case study in sparse probing. *arXiv preprint arXiv:2502.16681*, 2025.
- R. A. Harshman. Foundations of the PARAFAC procedure: Models and conditions for an explanatory multi-modal factor analysis. *UCLA Working Papers in Phonetics*, 16:1–84, 1970.
- P. O. Hoyer. Non-negative matrix factorization with sparseness constraints. *Journal of Machine Learning Research*, 5:1457–1469, 2004.
- T. G. Kolda and B. W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3): 455–500, 2009.
- J. B. Kruskal. Three-way arrays: rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics. *Linear Algebra and its Applications*, 18(2):95–138, 1977.
- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- J. Mairal, F. Bach, J. Ponce, and G. Sapiro. Online learning for matrix factorization and sparse coding. *Journal of Machine Learning Research*, 11:19–60, 2010.
- B. A. Olshausen and D. J. Field. Sparse coding with an overcomplete basis set: A strategy employed by V1? *Vision Research*, 37(23):3311–3325, 1997.
- A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, A. Pearce, C. Citro, E. Abrahams, A. Singer-Clark, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>, 2024.